PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective 5
3. Product Purpose 6
4. System Functionalities 7
5. User Stories & Use Cases 14
6. Features Included (Scope Definition) 19
7. Features Not Included (Scope Control) 21
8. Assumptions & Constraints 23
9. Risks & Questions Throughout Development 25

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: SME Finance Ops Copilot

Version: 1.0

1. Project Overview

- **Problem Statement:**
  Small and medium businesses (SMEs) often struggle with the manual administration of back-office finance operations, specifically managing reimbursements, petty cash, vendor invoice approvals, and internal payment requests. Currently, these critical workflows are handled through a highly fragmented mix of communication channels, relying heavily on emails, chat messages, spreadsheets, and manual follow-ups.

  Because these finance operations naturally involve messy, unstructured inputs, repeated approvals, and real monetary consequences, this disjointed approach leads to significant operational friction. SMEs frequently face processing delays, weak auditability, an increased risk of duplicate submissions, and inconsistent decision-making across the organization. This friction is heavily compounded by common edge cases such as blurry or partially missing receipt images, conflicting amounts between a chat message and the attached receipt, or claimants forgetting to include department allocations, all of which require extensive manual intervention and clarification loops.

  Furthermore, when SMEs attempt to digitize these processes, existing automated solutions often fall short. They typically behave like simple expense OCR (Optical Character Recognition) tools or static automation scripts, failing to act as a true, stateful workflow engine. These legacy tools cannot effectively handle ambiguity, dynamically route policy exceptions (like expenses above approval thresholds or unsupported spend categories), or maintain a continuous audit trail across multiple actors. Consequently,

© UMHackathon 2026
Email: umhackathon@um.edu.my

SMEs are left dealing with incomplete data and process failures, lacking the intelligent operations layer necessary to turn messy financial artifacts into structured workflows without undertaking the heavy burden of a massive ERP rollout.

- **Target Domain:** AI Systems & Agentic Workflow Automation, specifically targeting SME back-office finance operations and workflow orchestration.
- **Proposed Solution Summary:** The SME Finance Ops Copilot is a stateful workflow platform that acts as an intelligent operations layer for back-office finance case handling. It converts unstructured submissions (notes, receipts, invoices, PDFs, and image evidence) into governed, auditable workflow progression across requester, approver, finance-reviewer, and admin lanes. A GLM-powered intake layer performs extraction and routing analysis, but all downstream actions remain constrained by explicit backend transition rules and role-based authorization, so no critical financial state change is executed outside controlled workflow boundaries. When information is incomplete or unclear, the system opens clarification loops and pauses progression until the requester responds. Policy evaluation and lane controls detect risk patterns (for example blocking issues, duplicate signals, and finance-review triggers), route cases to approval or finance review as needed, and support delegation, reassignment, and exception handling. The platform also supports export-ready progression with recoverable exception paths, analytics for approval and finance queues, and complete traceability through workflow transitions and audit events for every key decision point.

© UMHackathon 2026

2. Background & Business Objective

- **Background of the Problem:** Traditionally, small and medium enterprises (SMEs) have managed critical financial workflows such as reimbursements, petty cash, invoice approvals, and internal payment requests through a highly fragmented mix of email, chat, spreadsheets, and manual follow-ups. Because these financial operations naturally involve messy, unstructured inputs and complex exceptions, this disjointed approach inherently leads to processing delays, weak auditability, the risk of duplicate submissions, and inconsistent decision-making.
- **Importance of Solving This Issue:** Resolving this friction delivers immediate, tangible business value by drastically reducing the processing time for finance and administrative staff while shortening reimbursement cycles, which directly improves the employee experience. Furthermore, it creates a clean, transparent approval trail that ensures strong audit readiness. Ultimately, solving this problem empowers SMEs to digitize and govern their finance operations seamlessly, completely bypassing the heavy financial burden and technical complexity of rolling out a large-scale Enterprise Resource Planning (ERP) system.
- **Strategic Fit / Impact:** This solution functions as a true stateful workflow engine rather than a simple chatbot or static script. It establishes GLM as an irreplaceable reasoning core, relying on the model to drive classification, clarification, routing, and exception handling across multiple workflow stages. Long-term, this initial implementation establishes a scalable architectural foundation designed to become the central finance operations backbone for SMEs, with a clear strategic roadmap to eventually expand into advanced treasury intelligence and liquidity command modules.

© UMHackathon 2026

3. Product Purpose

3.1. Main Goal of the System: To provide a stateful, agentic AI workflow engine that serves as an active operations layer for SME back-office finance workflows. The system aims to convert messy, unstructured financial inputs (such as mixed receipts, invoices, and natural-language chat instructions) into structured, highly auditable workflows, actively driving cases through intake, AI-led clarification, policy review, human approval, and final accounting export.

3.2. Intended Users (Target Audience):

- Employees (Requesters): Individuals who submit expense claims, petty cash reimbursements, or internal payment requests and need a frictionless, conversational intake process.
- Managers (Approvers): Team leaders responsible for reviewing team spend who need clear, summarized rationales to approve, reject, or request more information on a case.
- Finance and Admin Staff: Back-office professionals who manage the system's policy configurations, handle flagged or high-risk exceptions, and rely on the export-ready records for accounting.
- SME Founders: Business leaders overseeing cash movement and operational discipline who benefit from clean approval trails and automated exception handling without the need to deploy a heavy ERP system.

© UMHackathon 2026

4. System Functionalities

4.1. Description

The system operates as a Harness-driven finance operations control plane that transforms unstructured financial inputs into structured, auditable workflows through staged orchestration. Using a Case-to-Workflow execution model, the system captures user-submitted data such as receipts, invoices, and free-text notes, and converts them into persistent, traceable cases.

Through a sequence of explicit runtime stages (Intake, Extraction, AI Reasoning, Clarification, Policy Evaluation, Approval, Finance Review, Export, and Exception Handling), the system dynamically processes each case while maintaining full visibility and control. Intelligence is embedded via a governed AI layer that performs structured data extraction, confidence scoring, anomaly detection, and context-aware clarification generation, ensuring incomplete or ambiguous inputs are resolved interactively.

The system integrates a policy-driven decision engine that evaluates extracted data against configurable rules such as approval thresholds, required fields, and duplicate detection. Based on these evaluations, cases are automatically routed to the appropriate workflow path, including approval queues or finance review channels.

Human users remain actively involved through control gates, where approvers and finance reviewers validate, approve, reject, or request additional information. Each decision is supported by AI-generated rationale summaries and complete data provenance, ensuring transparency and accountability.

Additionally, the system emphasizes end-to-end auditability, where every action, system transition, AI output, and user interaction is recorded as part of a structured timeline. This enables full traceability from initial submission to final resolution.

© UMHackathon 2026

The platform also incorporates resilient exception handling, where failures such as data inconsistencies, extraction errors, duplicate submissions, or export issues are surfaced as recoverable workflow states. Users are provided with clear resolution paths, allowing the system to maintain operational continuity without silent failures.

Overall, the system functions as a modular, extensible workflow runtime for finance operations, enabling organizations to standardize and automate processes such as expense claims, invoice approvals, and payment requests while ensuring governance, explainability, and human oversight.

4.2. Key Functionalities:

- **Vibe Designing:** Transforms human-language financial requests like submitting a taxi receipt for client meetings into structured, workflow-ready cases. The system interprets intent, identifies relevant financial relationships such as amount, purpose and category, and organizes them into a normalized format suitable for downstream processing and decision-making.
- **Conversational Editing:** Enables continuous refinement of a case through interactive dialogue between the user and the system. Users can respond to AI-generated clarification questions or provide additional details via text input which allows the system to dynamically update extracted data, resolve ambiguities and improve case completeness without restarting the process.
- **Versioning & Infinite Canvas:** Maintains a complete history of case changes including AI extraction results, user inputs, policy evaluations and decision updates. Each modification is recorded as a new state in the case lifecycle, enabling traceability, rollback capability and visibility into how the case evolves across multiple workflow stages.
- **Export-Component Based:** Converts finalized cases into standardized, accounting-ready formats for integration with external financial systems. The system ensures all required fields are validated and complete before generating export payloads, while preserving

© UMHackathon 2026

structured data, audit logs and compatibility with downstream tools such as ERP or reporting systems.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.3. AI Model & Prompt Design

4.3.1. Model Selection: Do state the LLM used (Z AI GLM) along with justification why and what capabilities made it the right choice for the product's core functionalities.

The system uses **Z AI GLM** as the primary large language model for finance-intake reasoning and structured extraction. In the current backend runtime, the provider is invoked through the Z.AI chat-completions endpoint (default model: "glm-5.1") with "response_format: json_object". Prompt constraints require a strict two-object payload ("extraction", "decision") that can be mapped directly into workflow execution data. This design targets deterministic machine consumption rather than free-form generation, converting mixed evidence (request notes + OCR/document text) into typed fields, confidence, provenance, clarification questions, and a recommended transition intent.

Z AI GLM was selected because its operational characteristics align with the product's critical path. First, instruction adherence for schema-shaped outputs reduces post-processing ambiguity and simplifies parser normalization. Second, the model performs robustly on semi-structured, heterogeneous finance evidence, including incomplete text and low-signal notes, which is common in production case intake. Third, it provides adequate reasoning quality for classification-style control decisions (for example, "POLICY_REVIEW" versus "AWAITING_REQUESTER_INFO") while remaining compatible with stateless API orchestration and service-level timeout/fallback handling.

From an architecture perspective, GLM enables high automation while preserving control-plane safety. Raw model output is parsed, normalized, and validated against shared schemas before any state transition is evaluated by the backend workflow engine. The resulting pattern is AI-assisted extraction under deterministic guardrails: model-generated reasoning improves throughput and extraction coverage, while typed contracts and explicit transition logic keep runtime behavior auditable, testable, and stable under finance-operations constraints.

© UMHackathon 2026

4.3.2. **Prompting Strategy:**

The implemented prompting approach is a constrained zero-shot, single-call extraction-and-decision pattern. The backend sends one system instruction plus one structured user payload with JSON-serialized case context, then requires a machine-readable JSON response. This is not a few-shot and not a multi-turn agent loop, instead reliability is achieved through strict output constraints, low sampling variance with `temperature` set to `0.1`, and deterministic post-processing.

The system prompt defines both task and safety boundaries, extract finance-relevant fields from notes and artifact evidence, return only “extraction” and “decision”, and avoid inventing values when evidence quality is low. When OCR text is empty or weak or extraction warnings exist, the prompt explicitly instructs the model to lower confidence, omit unsupported fields, and generate clarification questions. This converts uncertain inputs into actionable workflow behavior with “AWAITING_REQUESTER_INFO” rather than hallucinated certainty.

This strategy fits the use case because the product's core requirement is operationally safe structured reasoning, not open-ended conversation. A zero-shot constrained format minimizes token overhead, keeps latency predictable, and integrates cleanly with synchronous API orchestration. Downstream normalization accepts minor key variations (“nextState”, “next_state”, “nextStep”) and applies schema validation plus fallback defaults, so the model can remain flexible at the language layer while the workflow engine remains deterministic at the control layer.

© UMHackathon 2026

4.3.3. Context & Input Handling

The system accepts mixed unstructured input from requester notes and uploaded artifacts. Notes are validated by schema and capped at 5000 characters on case submission. File uploads for production intake use multipart handling with a hard per-file cap of 25 MB on the artifact upload endpoint. Debug OCR routes apply separate limits and allow up to 10 files with 10 MB per file for OCR and 25 MB per file for PDF text extraction.

Oversized handling is implemented as hard rejection at the API boundary, not silent truncation. When an uploaded file exceeds the configured multipart size cap, the upload request is rejected by the interceptor layer and the artifact is not accepted for processing. For structured text fields, schema validation rejects values above declared maxima such as 5000 characters for “notes” and 2000 characters for decision reasons and question responses.

Text extraction behavior depends on artifact type. Plain text-like files are intentionally truncated to the first 24000 characters before persistence and AI intake, which prevents unbounded prompt growth from large text payloads. PDF and image extraction do not currently apply an explicit post-extraction character cap in service code, and no chunking pipeline is used for multi-pass model calls. Unsupported binary formats are accepted as artifacts but produce empty extracted text with extraction warnings, which then pushes the workflow toward clarification instead of fabricated field values.

For inputs that exceed model context limits after preprocessing, no dedicated chunk-and-merge path is implemented in the current release. The provider call can fail and surface an API error unless the failure matches configured fallback conditions such as timeout or upstream server errors. In short, the current design prefers strict boundary limits and deterministic rejection behavior over lossy implicit compression of user evidence.

© UMHackathon 2026

4.3.4. Fallback & Failure Behavior

The system handles off-topic or unusable model output through layered constraints before workflow decisions are persisted. The provider request enforces JSON-object output, the prompt requires strict “extraction” and “decision” objects, and backend normalization maps minor key drift into canonical fields. Parsed content is validated against shared schemas, so malformed or structurally invalid output is rejected instead of being treated as trusted decision data.

Hallucination risk is reduced at prompt and control-flow levels. The prompt explicitly instructs the model to avoid inventing values when artifact evidence is weak and to create clarification questions when confidence is low. If evidence is missing or extraction warnings are present, the expected path becomes clarification-driven intake and state progression toward “AWAITING_REQUESTER_INFO” rather than forced advancement with fabricated fields.

Fallback behavior is implemented in the AI gateway service. When live Z.AI inference fails with timeout-like errors, connection reset signatures, or upstream status at or above 500, the gateway can fall back to the mock provider when “AI_FALLBACK_TO_MOCK” is enabled. Fallback provenance is written into “modelMetadata” with provider identity and failure reason so downstream audit surfaces can distinguish degraded inference from primary-model output.

Retry behavior is intentionally minimal in the current release. No multi-attempt retry loop with exponential backoff is implemented for the live provider path. If fallback is disabled or the failure does not match fallback criteria, the request fails and returns an API error to the caller. A dedicated AI-specific error state is not yet modeled as a separate workflow status.

Human escalation is provided through workflow operations rather than direct model override endpoints. Requesters can respond to generated clarification questions, approvers and finance reviewers can hold or reject cases, and admins can intervene through routing and policy controls. This design keeps AI outputs advisory within a governed human-in-the-loop process even when model quality degrades.

© UMHackathon 2026

5. User Stories & Use Cases

5.1. User Stories

Requester	I want to create a finance case with my workflow type so that the system starts the correct review path.
	I want to upload receipts, invoices, and supporting files in one form so that evidence is available for extraction and review.
	I want to submit notes in my own wording so that I can explain business context without filling rigid templates.
	I want to answer clarification questions after submission so that my case can continue when information is missing.
	I want to see case status and latest decisions in one detail page so that I know what action is needed next.
Approver	I want a queue of pending approval tasks with case context so that I can process decisions efficiently.
	I want to approve or reject with a decision reason so that governance decisions are traceable.
	I want to request more information from the requester so that I can avoid approving unclear evidence.
	I want delegated handling support for ownership changes so that tasks can continue when the original owner is unavailable.
Finance reviewer	I want a dedicated queue for escalated cases so that finance exceptions are handled separately from normal approvals.
	I want to approve, reject, send back, or assign reviews so that exception resolution follows policy and workload needs.
	I want to review metadata such as reason category and reconciliation status so that final decisions are documented consistently.
Admin	I want to configure policy thresholds and routing defaults so that decision flow aligns with current operating rules.
	I want to manage delegation and approval matrix settings so that review responsibility can be controlled across teams.

© UMHackathon 2026

Email: umhackathon@um.edu.my

	I want connector health visibility so that runtime readiness can be checked before operational use.

© UMHackathon 2026

Email: umhackathon@um.edu.my

5.2 Use Cases (Main Interactions)

5.2.1 Evidence to workflow flow requester submission to routed state

The requester opens the new case page, chooses workflow type, writes notes in natural language, and stages one or more real files using click upload, drag and drop, or paste. On submit, the client creates a draft case, uploads each file to the artifact upload endpoint, and then calls case submission. The backend validates that at least one artifact exists and blocks submission if extracted text is empty or extraction warnings indicate unreadable evidence. If blocked, a replacement draft case is created and the requester receives a guided re-upload message in the UI. If evidence passes quality gates, the backend transitions the case through submitted and intake processing, runs AI intake analysis, persists extraction output, and transitions to the next state based on decision intent. If the next state is policy review, policy routing runs immediately and can continue toward approval or finance review. This use case is the primary end to end intake interaction and is designed to transform unstructured input into an actionable governed workflow path.

5.2.2 Evidence to workflow flow requester submission to routed state

When AI intake or downstream reviewers identify missing evidence, the case enters an awaiting requester information state with open questions visible on case detail. The requester answers directly in the question response form, which sends structured responses to the question endpoint and refreshes case state. For approver initiated follow up, the approval request info action creates a new question and transitions the case to awaiting approver information response until the requester answers. For finance initiated follow up, send back transitions the case to awaiting requester information and create a finance review sourced question. After responses are provided, the workflow can re enter policy and continue normal routing. This use case ensures that incomplete submissions are recoverable through guided clarification instead of manual case recreation.

© UMHackathon 2026

5.2.3 Approval queue flow pending task to approved rejected delegated or info requested

Approvers open the approvals queue and receive task cards with case context, stage metadata, and action controls. Approve marks the task approved and either activates the next stage or transitions the case to approved and export ready when matrix requirements are complete. Reject marks the task rejected, cancels remaining stage tasks, and transitions the case to rejected. Request info captures the question, transitions the case to awaiting approver information response, and pushes follow up back to requester action. Delegate transfers ownership to another approver, records an audit event, and updates case assignment. The page also refreshes queue state after action completion so operators can continue processing without leaving the queue. This use case represents the core business control interaction for manager and approval lane actors.

5.2.4 Finance review flow escalated case to final resolution

Finance reviewers open the finance review queue for escalated cases and can assign owner, approve, reject, or send back with structured metadata such as reason category, coding decision, reconciliation status, reconciled amount, currency, annotation, and review note. Approve resolves the review, transitions case to approved, then marks export ready and ensures export record readiness. Reject resolves the review and transitions case to rejected. Send back resolves the review as sent back, transitions case to awaiting requester information, and creates a finance follow up question for requester response. Assign updates ownership and logs assignment for traceability. This use case provides manual exception handling with stronger finance specific control data before final export.

© UMHackathon 2026

5.2.5 Export and recovery flow export ready case to closed or recoverable exception

From case detail, authorized users can trigger export using the export action control. Successful export returns updated export record and advances case toward exported and closed states. If export fails, the workflow enters a recoverable exception rather than terminal loss, allowing controlled remediation. Finance reviewer or admin users can trigger recovery action on eligible cases, which transitions from recoverable exception back into policy review and re-run routing. This creates a governed retry pathway for operational continuity when connector or processing failures occur.

5.2.6 Artifact retry and re ingestion flow improved file quality to refreshed intelligence

If an artifact extraction fails or quality is poor, operators can trigger retry processing from case detail. The retry endpoint reprocesses the artifact and when successful triggers intake refresh logic for eligible workflow states. The refresh path computes an evidence digest and skips unnecessary AI churn when extracted text is unchanged. When evidence content changes, AI re analysis and policy rerouting can update case posture while preserving transition and audit visibility. This use case supports iterative evidence quality improvement without discarding the original case trail.

5.2.7 Operational monitoring flow dashboard to targeted lane action

Operations users open a dashboard to monitor open workload, cases needing action, lane counts, workflow distribution, intake stage backlog, and recent cases. From this view users can jump directly into approvals, finance review, new case creation, or case detail investigation. Queue analytics on approvals and finance pages provide throughput and bottleneck signals to guide staffing and assignment decisions. This use case connects high level operational visibility with direct action entry points for daily control room style workflow management.

© UMHackathon 2026

6. Features Included (Scope Definition)

6.1. End to end finance case lifecycle in one product

The system handles a case from creation to closure in one continuous flow. Users can submit a request, progress through review lanes, trigger export, and verify final state without moving across disconnected tools.

6.2. AI assisted evidence to decision pipeline

Unstructured notes and document text are transformed into structured extraction output that includes fields, confidence, provenance, open questions, and routing intent. This feature turns raw evidence into actionable workflow input rather than static storage.

6.3. Smart clarification loop instead of hard failure

When evidence is weak or incomplete, the system asks targeted follow up questions and pauses the case in a requester information lane. After answers are submitted, the same case continues through normal routing without manual re-creation.

6.4. Dual control queues for approval and finance exception handling

The product provides two operational queues with different control intents. Approval queue manages business sign off actions, while finance review queue handles escalations, send back actions, assignment, and risk resolution before export.

6.5. Unified case intelligence view

Each case has a single detail surface that combines extracted fields, policy output, review decisions, export readiness, evidence status, open questions, transitions, and audit events. This removes context switching during investigation and action.

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.6. Evidence aware artifact operations

Users can upload real files through click, drag and drop, or paste interactions. The backend processes text, PDF, and image evidence, supports preview for stored files, and enables artifact reprocessing when extraction quality needs improvement.

6.7. Recoverable exception path for failed exports

Export failures do not terminate the case blindly. The system records failed export state, marks the case as a recoverable exception, and supports manual recovery routing so operations can continue from a controlled checkpoint.

6.8. Live operations visibility dashboard

The dashboard provides real time operational visibility including open workload, cases needing user action, lane specific counts, workflow distribution, and intake stage monitoring. This supports daily control room style usage for team leads and operators.

6.9. Policy and routing control panel for admins

Admins can tune policy thresholds and routing defaults at runtime through dedicated settings forms. Connector health cards also expose integration readiness so admin users can validate environment posture during demos or operations.

6.10. Built in human governance across all AI outcomes

AI output does not auto finalize finance outcomes. Human actors in requester, approver, finance reviewer, and admin lanes can validate, override, request more information, reject, or re-route cases at critical checkpoints.

© UMHackathon 2026

Email: umhackathon@um.edu.my

7. Features Not Included (Scope Control)

7.1. Full production integration posture is not included

The current system is designed for controlled development and demo operation. Several connectors run with environment specific runtime configurations, including AI fallback paths, storage flow variants, and demo oriented auth behavior. Full production hardening for all integrations is not part of this release scope.

7.2. Full multi tier approval matrix behavior is not included

Approval stage controls exist, but complete enterprise depth approval matrix behavior is not fully implemented in the current release. Advanced matrix design patterns across all business scenarios remain in the future scope.

7.3. Full production integration posture is not included

Data structures reserve FX related fields, but end to end FX reconciliation logic with full calculation and settlement handling is not complete in current runtime behavior.

7.4. Full production integration posture is not included

The current implementation does not provide full accounting grade validation for net amount, tax amount, and gross amount consistency across all workflows and document conditions.

7.5. Advanced duplicate reasoning is not included

Duplicate detection support exists at a practical level, but richer duplicate intelligence and deeper evidence reasoning are not fully implemented in this release.

7.6. Dedicated chunk and merge AI processing for oversized context is not included

The current AI path does not implement multi pass chunking with merge logic for very large combined evidence contexts. Oversized behavior is controlled mainly through input limits, bounded extraction for text like files, and error handling paths.

© UMHackathon 2026

Email: umhackathon@um.edu.my

7.7. AI specific retry orchestration with backoff is not included

The current system supports provider fallback under selected failure conditions, but does not implement a dedicated multi attempt retry strategy with exponential backoff and policy tuned retry budgets.

7.8. Specialized AI hallucination adjudication module is not included

Hallucination risk is managed through prompt constraints, schema validation, and clarification loops. A separate adjudication service with confidence calibration models or secondary verifier models is not part of the current scope.

7.9. True real time collaborative editing is not included

The product supports multi user workflow participation through role based queues and actions, but does not provide real time concurrent co editing of the same form or shared cursor style collaboration.

7.10. Native mobile client applications are not included

Current delivery targets web application routes and API services. Dedicated iOS and Android native app experiences are outside this release scope.

7.11. Broad workflow family coverage beyond current finance lanes is not included

The current scope focuses on implemented finance case lanes and their supporting review flow. Additional domain families and broader enterprise process catalog coverage are planned as future expansion items.

7.12. Scope control summary

Current scope prioritizes a working end to end finance case pipeline with AI assisted intake, governed review lanes, export handling, and operational visibility. Features listed above are intentionally excluded to keep delivery focused, testable, and aligned to current project phase objectives.

© UMHackathon 2026

Email: umhackathon@um.edu.my

8. Assumptions & Constraints

8.1. LLM cost constraint

Cost estimation is based on one intake submission session with notes plus extracted evidence and one model response. A practical planning assumption is around 6000 to 18000 total tokens per active submission session depending on evidence volume and text quality. This estimate should be treated as a budgeting baseline and not as a billing exact value.

At least one design decision was made specifically to control inference spend at scale. The AI path uses a single constrained call for extraction and routing intent instead of multi turn agent loops. Additional cost control choices include bounded note length, bounded text extraction for text like files, strict JSON output shaping, and no default chunk and merge multi pass inference pipeline.

8.2. Technical constraints

The current delivery target is a web application with API services and shared contracts. Native mobile clients are not included. Full multi tier approval matrix depth, complete FX reconciliation behavior, and full net tax gross accounting validation are outside current scope. Oversized context handling does not include a dedicated chunk and merge reasoning pipeline in this release.

8.3. Performance Constraints

Performance is constrained by evidence size, extraction workload, and AI context size. File upload limits are enforced at API boundaries and oversized payloads are rejected. Text-like extraction is bounded to reduce unbounded prompt growth. Very large or low quality evidence sets can increase latency, increase failure probability, or require clarification loops before the workflow can progress.

© UMHackathon 2026

8.4. User input and human governance constraints

The product requires explicit user input to start and progress cases. The model does not autonomously initiate work without submitted notes and artifacts. Human actions remain mandatory at governance checkpoints including requester clarification, approval decisions, finance review decisions, and admin policy or routing control. This keeps final operational authority with human roles rather than with model output alone.

© UMHackathon 2026

9. Risks & Questions Throughout Development

9.1. Outcome similarity across different users and cases

When evidence quality is low or highly repetitive, model outputs can converge into similar reasoning summaries, extracted field patterns, or routing recommendations across unrelated cases. This can hide case specific nuance and reduce operator trust when different submissions appear to receive near identical AI posture. Current handling uses evidence provenance, open question generation, policy review, and human decision lanes to prevent blind execution from similar model text alone. A key open question is whether to add case specific differentiation checks, such as similarity scoring across recent reasoning summaries and escalation when overlap exceeds a threshold.

9.2. Workflow frame stability across state transitions

Complex cases can move through clarification, approval, finance review, export, and recovery with repeated loops, and stability risk appears when state transitions or queue visibility become inconsistent after edge path execution. If state framing drifts, operators may act in the wrong lane or miss required manual steps. Current handling guards transition writes, rejects stale transitions, and exposes transitions with audit events in case detail for verification. A central open question is whether to add stronger automated regression coverage for multi loop scenarios, especially combined request info loops and export recovery loops in one continuous case timeline.

9.3. Evidence interpretation risk for weak or ambiguous documents

Uploaded evidence can be blurry, incomplete, unsupported, or text sparse, and in these conditions the model and extraction services may not contextualize the document sufficiently for confident routing. Incorrect interpretation can produce wrong fields, delayed review, or unnecessary escalation load. Current handling discourages unsupported field invention through prompt controls, keeps extraction warnings visible, allows confidence reduction, and routes clarification questions back to requester input. An open question is whether confidence thresholds should be made more explicit for transition gating so low quality interpretation always triggers a stricter manual review path.

© UMHackathon 2026

Email: umhackathon@um.edu.my

9.4. Cost versus quality trade off under scale

As evidence volume grows, maintaining extraction quality while keeping inference cost and latency within budget becomes harder. High latency reduces usability and high inference spend reduces scalability for sustained operational adoption. Current handling uses a single constrained call pattern with bounded inputs and deterministic parsing to limit uncontrolled token growth. A forward looking question is whether selective caching, adaptive context compression, or tiered model routing should be introduced for high volume deployment phases.

9.5. Human action bottleneck risk

The system intentionally keeps humans at key checkpoints, and under high throughput this can create queue backlogs in approver and finance reviewer lanes. Backlog accumulation increases turnaround time and can reduce the operational value of fast AI intake. Current handling uses queue analytics and dashboard visibility to signal pending work, blocked items, and unassigned reviews. An open question is whether automated workload balancing or stronger assignment policies should be added to prevent recurring bottlenecks across specific reviewer groups.

© UMHackathon 2026